

**"AÑO DEL BICENTENARIO, DE LA CONSOLIDACIÓN DE
NUESTRA INDEPENDENCIA, Y DE LA CONMEMORACIÓN DE LAS
HEROICAS BATALLAS DE JUNÍN Y AYACUCHO"**

UNIVERSIDAD PERUANA LOS ANDES



UPLA

**ESCUELA PROFESIONAL DE INGENIERIA DE
SISTEMAS**

ASIGNATURA:

BASE DE DATOS 2

TEMA:

"RESOLUCION DE EJERCICIOS SEMANA 14"

DOCENTE : RAUL ENRIQUE FERNANDEZ BEJARANO

ESTUDIANTE : CUBA VILLANUEVA JAVIER AMILCAR

CICLO : 5to

Huancayo – 2024

TEMA 1 – CREACIÓN DE JOBS Y ALERTS CON SQL SERVER AGENT

Proyecto 1:

1. Enunciado del ejercicio

La Oficina de Grados y Títulos requiere que diariamente a las 08:00 AM se genere automáticamente un reporte con la cantidad de trámites registrados el día anterior en la base de datos **GradosTitulos**.

La ejecución debe registrarse en una tabla de auditoría y programarse mediante SQL Server Agent.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

CREATE TABLE dbo.AuditoriaJobs
(
    IdAuditoria INT IDENTITY(1,1) PRIMARY KEY,
    FechaEjecucion DATETIME DEFAULT GETDATE(),
    TotalTramites INT
);
GO

USE msdb;
GO

EXEC sp_add_job
    @job_name = 'JOB_ReporteTramitesDiario';
GO

EXEC sp_add_jobstep
    @job_name = 'JOB_ReporteTramitesDiario',
    @step_name = 'Generar Reporte',
    @subsystem = 'TSQL',
    @database_name = 'GradosTitulos',
    @command = '

INSERT INTO dbo.AuditoriaJobs(TotalTramites)

SELECT COUNT(*)

FROM Tramite.Tramite

WHERE FechaInicio = CAST(GETDATE()-1 AS DATE)

';
GO
```

3. Justificación Técnica

Se implementó un Job de SQL Server Agent para automatizar la generación del reporte diario de trámites. El Job ejecuta una consulta sobre la tabla Tramite.Tramite y almacena el resultado en una tabla de auditoría. La programación a las 08:00 AM garantiza que el reporte se genere sin intervención manual y quede disponible para consulta posterior.

4. Explicación de las buenas prácticas utilizadas

- Uso de SQL Server Agent para automatización.
- Registro de resultados en una tabla de auditoría.

- Separación entre Job, Step y Schedule.
- Nombres descriptivos para los objetos creados.
- Programación centralizada y fácil de modificar.

Proyecto 2:

1. Enunciado del ejercicio

Se requiere automatizar la actualización semanal del campo FechaUltMod para los trámites que permanezcan en estado EN_PROCESO, registrando cada ejecución mediante SQL Server Agent.

2. Script de la solución en T-SQL

```
--1.2

CREATE TABLE dbo.HistorialActualizacion
(
    IdHistorial INT IDENTITY(1,1),
    FechaProceso DATETIME DEFAULT GETDATE(),
    RegistrosActualizados INT
);
GO
USE msdb;
GO

EXEC sp_add_job
    @job_name='JOB_ActualizarTramites';
GO
EXEC sp_add_jobstep
    @job_name='JOB_ActualizarTramites',
    @step_name='Actualizar FechaUltMod',
    @subsystem='TSQL',
    @database_name='GradosTitulos',
    @command='

    UPDATE T
    SET FechaUltMod = GETDATE()

    FROM Trámite.Tramite T
    INNER JOIN Catalogo.EstadoTrámite E
    ON T.IdEstadoTrámite = E.IdEstadoTrámite

    WHERE E.Descripcion = ''EN_PROCESO'';

    INSERT INTO dbo.HistorialActualizacion
    VALUES(GETDATE(), @@ROWCOUNT);

';
GO
```

3. Justificación Técnica

La automatización semanal evita que los administradores deban actualizar manualmente los registros. SQL Server Agent permite ejecutar el proceso de manera periódica y registrar la cantidad de filas modificadas para fines de control y auditoría.

4. Explicación de las buenas prácticas utilizadas

- Automatización mediante SQL Server Agent.
- Registro histórico de ejecuciones.
- Uso de @@ROWCOUNT para controlar cambios.
- Actualización únicamente de registros necesarios.
- Código reutilizable y fácil de mantener.
- Programación periódica centralizada.

Proyecto 3:

1. Enunciado del ejercicio

Se requiere diseñar un Job que elimine mensualmente los registros antiguos de la tabla `Tramite.LogTemporal`. Antes de eliminarlos, los registros con más de 180 días deberán almacenarse en una tabla histórica.

2. Script de la solución en T-SQL

```
76  --1.3
77
78  CREATE TABLE Tramite.LogTemporalHistorico
79  (
80      IdLog INT,
81      FechaRegistro DATETIME,
82      Descripcion VARCHAR(500)
83  );
84  GO
85
86  CREATE OR ALTER PROCEDURE Tramite.SP_DepurarLogTemporal
87  AS
88  BEGIN
89
90      SET NOCOUNT ON;
91
92      BEGIN TRY
93
94          BEGIN TRANSACTION;
95
96          INSERT INTO Tramite.LogTemporalHistorico
97          (
98              IdLog,
99              FechaRegistro,
100             Descripcion
101          )
102          SELECT
103              IdLog,
104              FechaRegistro,
105              Descripcion
106          FROM Tramite.LogTemporal
107          WHERE FechaRegistro < DATEADD(DAY,-180,GETDATE());
108
109          DELETE FROM Tramite.LogTemporal
110          WHERE FechaRegistro < DATEADD(DAY,-180,GETDATE());
111
112          COMMIT TRANSACTION;
113
114      END TRY
115
116      BEGIN CATCH
117
118          ROLLBACK TRANSACTION;
```

3. Justificación Técnica

La solución implementa una política de depuración automática para reducir el crecimiento de las tablas temporales. Antes de eliminar los registros, estos son almacenados en una tabla histórica para conservar la trazabilidad de la información. El uso de transacciones garantiza la integridad de los datos durante el proceso.

4. Explicación de las buenas prácticas utilizadas

- Uso de transacciones para garantizar consistencia.
- Implementación de TRY...CATCH para manejo de errores.

- Conservación de datos históricos antes de eliminarlos.
- Automatización mediante SQL Server Agent.
- Procedimiento almacenado reutilizable.

TEMA 2 – USO DE DATABASE MAINTENANCE PLANS

Proyecto 1:

1. Enunciado del ejercicio

La Oficina de Tecnologías de Información requiere implementar un **Plan de Mantenimiento** que realice diariamente un respaldo completo de la base de datos **GradosTitulos**. El proceso debe generar el archivo de respaldo, verificar su integridad y dejar la configuración lista para ser programada mediante SQL Server Agent.

2. Script de la solución en T-SQL

```
--2.1
USE master;
GO

BACKUP DATABASE GradosTitulos
TO DISK = 'C:\Backups\GradosTitulos\GradosTitulos_Full.bak'
WITH
FORMAT,
INIT,
COMPRESSION,
STATS = 10;
GO

RESTORE VERIFYONLY
FROM DISK = 'C:\Backups\GradosTitulos\GradosTitulos_Full.bak';
GO

SELECT
    database_name,
    backup_start_date,
    backup_finish_date,
    type,
    physical_device_name
FROM msdb.dbo.backupset bs
INNER JOIN msdb.dbo.backupmediafamily bm
ON bs.media_set_id = bm.media_set_id
WHERE database_name = 'GradosTitulos'
ORDER BY backup_finish_date DESC;
GO
```

Resultados

Mensajes

	database_name	backup_start_date	backup_finish_date	type	physical_device_name
1	GradosTitulos	2026-07-13 22:28:34.000	2026-07-13 22:28:34.000	D	C:\Backups\GradosTitulos\GradosTitulos_Full.bak

3. Justificación Técnica

Se implementó un respaldo completo utilizando la instrucción **BACKUP DATABASE**, incorporando la opción **COMPRESSION** para reducir el tamaño del archivo y disminuir el tiempo de ejecución. Posteriormente, se verificó la integridad del respaldo mediante **RESTORE VERIFYONLY**, asegurando que el archivo pueda utilizarse en un proceso de restauración.

4. Explicación de las buenas prácticas utilizadas

- Se utilizó un respaldo **FULL** para garantizar una copia completa de la base de datos.
- Se habilitó la compresión para optimizar espacio en disco.
- Se verificó la integridad del respaldo con **RESTORE VERIFYONLY**.
- Se consultó el historial de respaldos para validar su correcta ejecución.
- Se recomendó una programación diaria mediante SQL Server Agent.

Proyecto 2:

1. Enunciado del ejercicio

La base de datos **GradosTitulos** presenta una disminución en el rendimiento debido a la fragmentación de los índices en las tablas **Persona**, **Tramite**, **TrabajoInvestigacion** y **Sustentacion**.

Se requiere implementar un **Plan de Mantenimiento** que reorganice periódicamente los índices de estas tablas y deje configurada su ejecución automática mediante **SQL Server Agent**.

2. Script de la solución en T-SQL

```
163  --2.2
164
165  USE GradosTitulos;
166  GO
167
168  SELECT
169      OBJECT_NAME(ps.object_id) AS Tabla,
170      i.name AS Indice,
171      ps.avg_fragmentation_in_percent AS Fragmentacion
172  FROM sys.dm_db_index_physical_stats
173  (
174      DB_ID(),
175      NULL,
176      NULL,
177      NULL,
178      'LIMITED'
179  ) ps
180  INNER JOIN sys.indexes i
181  ON ps.object_id = i.object_id
182  AND ps.index_id = i.index_id
183  WHERE OBJECT_NAME(ps.object_id) IN
184  (
185      'Persona',
186      'Tramite',
187      'TrabajoInvestigacion',
188      'Sustentacion'
189  )
190  ORDER BY Tabla;
191  GO
192
193  ALTER INDEX ALL
194  ON Academico.Persona
195  REORGANIZE;
196  GO
```

Resultados		Mensajes	
	Tabla	Indice	Fragmentacion
1	Persona	PK_Persona	0
2	Persona	UQ_Persona_DNI	0
3	Persona	IX_Persona_DNI	0
4	Persona	IX_Persona_Tipo	0
5	Persona	IX_Persona_Apellidos	0
6	Sustentacion	PK_Sustentacion	0
7	Sustentacion	UQ_Sust_Trabajo_Intento	0
8	Sustentacion	IX_Sust_Trabajo	0
9	TrabajoInvestigacion	PK_TrabajoInvestigacion	0

3. Justificación Técnica

Se utilizó la instrucción ALTER INDEX ... REORGANIZE para reducir la fragmentación de los índices sin bloquear completamente las tablas durante el proceso. Antes y después del mantenimiento se consultó la vista dinámica sys.dm_db_index_physical_stats para verificar el porcentaje de fragmentación y comprobar la efectividad del proceso

4. Explicación de las buenas prácticas utilizadas

- Se verificó el estado de los índices antes y después del mantenimiento.
- Se utilizó **REORGANIZE**, recomendado para índices con fragmentación moderada.
- Se aplicó el mantenimiento únicamente a las tablas más utilizadas de la base de datos.
- Se empleó la vista dinámica sys.dm_db_index_physical_stats para obtener información precisa sobre la fragmentación.

Proyecto 3:

1. Enunciado del ejercicio

La base de datos **GradosTitulos** requiere un plan de mantenimiento que optimice periódicamente el rendimiento de los índices. El proceso debe identificar el nivel de fragmentación de los índices, reorganizarlos cuando la fragmentación sea menor al **30%**, reconstruirlos cuando sea mayor al **30%** y actualizar las estadísticas para mejorar los planes de ejecución de SQL Server.

2. Script de la solución en T-SQL

```

90 USE GradosTitulos;
91 GO
92
93 SELECT
94     OBJECT_NAME(ps.object_id) AS Tabla,
95     i.name AS Indice,
96     ps.avg_fragmentation_in_percent AS Fragmentacion,
97     ps.page_count AS Paginas
98 FROM sys.dm_db_index_physical_stats
99 (
100     DB_ID(),
101     NULL,
102     NULL,
103     NULL,
104     'LIMITED'
105 ) ps
106 INNER JOIN sys.indexes i
107 ON ps.object_id = i.object_id
108 AND ps.index_id = i.index_id
109 WHERE ps.index_id > 0
110 ORDER BY Fragmentacion DESC;
111 GO
112
113 ALTER INDEX ALL
114 ON Academico.Persona
115 REORGANIZE;
116 GO
117
118 ALTER INDEX ALL
119 ON Tramite.Tramite
120 REORGANIZE;
121 GO

```

	Tabla	Indice	Fragmentacion	Paginas
1	Pago	PK_Pago	0	0
2	LogRetencion	PK_LogReten__3214EC0713A47049	0	0
3	Fotografia	PK_Fotografia	0	0
4	TrabajoInvestigacion	PK_TrabajoInvestigacion	0	0
5	TrabajoInvestigacion	IX_Trabajo_Tramite	0	0
6	TrabajoInvestigacion	IX_Trabajo_Tipo	0	0
7	HistoricoConsultasLentas	PK__Historic__9CC7DBB4128AA46D	0	0
8	HistorialMonitoreo	PK__Historia__9CC7DBB44DBA1AF5	0	0
9	HistorialMantenimiento	PK__Historia__DD1C441772240141	0	1

3. Justificación Técnica

Se implementó un plan de mantenimiento basado en las recomendaciones de Microsoft para la administración de índices en SQL Server. Mediante la vista dinámica sys.dm_db_index_physical_stats se identificó el nivel de fragmentación de los índices. Para índices con fragmentación moderada se aplicó ALTER INDEX ... REORGANIZE, mientras que para aquellos con fragmentación superior al 30% se utilizó ALTER INDEX ... REBUILD

4. Explicación de las buenas prácticas utilizadas

- Se evaluó el estado de los índices antes de realizar cualquier mantenimiento.
- Se aplicó **REORGANIZE** únicamente a índices con fragmentación moderada para reducir el impacto sobre el servidor.
- Se utilizó **REBUILD** para índices con alta fragmentación, mejorando su estructura y rendimiento.
- Se actualizaron las estadísticas después del mantenimiento para optimizar los

planes de ejecución.

- Se empleó la vista dinámica sys.dm_db_index_physical_stats para obtener información confiable sobre la fragmentación.

TEMA 3. Automatización con T-SQL y PowerShell

Proyecto 1:

1. Enunciado del Ejercicio Crear un procedimiento almacenado que genere automáticamente un respaldo completo y posteriormente sea ejecutado mediante PowerShell.

2. Script de la solución

```
233 --3.1
234
235 USE GradosTitulos;
236 GO
237
238 CREATE OR ALTER PROCEDURE dbo.SP_GenerarBackupCompleto
239 AS
240 BEGIN
241     SET NOCOUNT ON;
242     DECLARE @Ruta NVARCHAR(255) = 'C:\Backups\GradosTitulos_Full_PS.bak';
243
244     BACKUP DATABASE GradosTitulos
245     TO DISK = @Ruta
246     WITH FORMAT, INIT, NAME = 'Backup Full via PowerShell';
247 END;
248 GO
```

```
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Write-Host "El respaldo completo se ha ejecutado exitosamente a través de PowerShell."
ndColor Green
El respaldo completo se ha ejecutado exitosamente a través de PowerShell.
PS C:\WINDOWS\system32>
```

3. Justificación Técnica de la solución aplicada Se divide la solución en dos capas. La capa de base de datos (Stored Procedure) encapsula la lógica pura del BACKUP DATABASE, asegurando que SQL Server maneje los permisos y rutas internamente. La capa de sistema operativo (PowerShell) utiliza el cmdlet Invoke-Sqlcmd para establecer la conexión al motor y disparar la ejecución sin necesidad de abrir herramientas gráficas como SSMS.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Desacoplamiento:** Al no escribir el código T-SQL de backup directamente dentro de PowerShell, permitimos que el DBA pueda modificar la ruta de destino o los parámetros de retención en el Procedimiento Almacenado sin tener que tocar los scripts del sistema operativo.

Proyecto 2:

1. Enunciado del Ejercicio Automatizar la exportación diaria de los trámites aprobados hacia un archivo CSV utilizando PowerShell.

2. Script de la solución en PowerShell

```

PS C:\WINDOWS\system32> $Instancia = "localhost"
PS C:\WINDOWS\system32> $BaseDatos = "GradosTitulos"
PS C:\WINDOWS\system32> $RutaArchivoCSV = "C:\Reportes\TramitesAprobados_$(Get-Date -Format 'yyyyMMdd').csv"
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> # Consulta T-SQL para obtener trámites aprobados (Asumiendo IdEstado = 5)
PS C:\WINDOWS\system32> $ConsultaSQL = @"
> SELECT
>     IdTramite,
>     FechaInicio,
>     Observaciones
> FROM Tramite.Tramite
> WHERE IdEstadoTramite = 5;
> "@
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> # Ejecutar consulta y canalizar el resultado hacia Export-Csv
PS C:\WINDOWS\system32> Invoke-Sqlcmd -ServerInstance $Instancia -Database $BaseDatos -Query $ConsultaSQL |
> Export-Csv -Path $RutaArchivoCSV -NoTypeInformation -Encoding UTF8
Invoke-Sqlcmd : El comando 'Invoke-Sqlcmd' se encontró en el módulo 'SQLPS', pero el módulo no pudo cargarse.
Para obtener más información, ejecute 'Import-Module SQLPS'.
En línea: 1 Carácter: 1
+ Invoke-Sqlcmd -ServerInstance $Instancia -Database $BaseDatos -Query ...
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (Invoke-Sqlcmd:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CouldNotAutoloadMatchingModule
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Write-Host "Exportación a CSV generada en: $RutaArchivoCSV" -ForegroundColor Cyan
Exportación a CSV generada en: C:\Reportes\TramitesAprobados_20260713.csv

```

3. Justificación Técnica de la solución aplicada Se aprovecha el "Pipeline" (|) característico de PowerShell. El cmdlet Invoke-Sqlcmd ejecuta el SELECT y devuelve los datos como objetos de PowerShell. Estos objetos se envían inmediatamente al cmdlet Export-Csv. El parámetro -NoTypeInformation se incluye porque, por defecto, PowerShell añade una primera línea con metadatos del objeto que arruina el formato estándar del CSV.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Nomenclatura Dinámica:** Se incluyó \$(Get-Date -Format 'yyyyMMdd') en el nombre del archivo para que cada ejecución diaria genere un archivo nuevo en lugar de sobrescribir el del día anterior, creando un histórico automático.

Proyecto 3:

1. Enunciado del Ejercicio Desarrollar un proceso automático que consulte diariamente la cantidad de trámites, genere un archivo Excel y almacene una copia en una carpeta de respaldo.

2. Script de la solución en PowerShell

```

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows
PS C:\Users\Usuario> # Nota: Requiere haber instalado el módulo previamente: Install-Module -Name ImportExcel
PS C:\Users\Usuario>
PS C:\Users\Usuario> $Instancia = "localhost"
PS C:\Users\Usuario> $RutaExcelPrincipal = "C:\ReportesOficiales\ResumenTramites.xlsx"
PS C:\Users\Usuario> $CarpetaRespaldo = "C:\RespalDOSExcel\"
PS C:\Users\Usuario> $RutaExcelCopia = $CarpetaRespaldo + "Resumen_$(Get-Date -Format 'yyyyMMdd_HH:mm:ss').xlsx"
PS C:\Users\Usuario>
PS C:\Users\Usuario> # 1. Consulta en T-SQL
PS C:\Users\Usuario> $Consulta = "SELECT CAST(GETDATE() AS DATE) AS FechaConsulta, COUNT(*) AS CantidadTotalTramites FROM
M GradosTitulos.Tramite.Tramite;"
PS C:\Users\Usuario>
PS C:\Users\Usuario> # 2. Ejecutar y exportar a Excel
PS C:\Users\Usuario> Invoke-Sqlcmd -ServerInstance $Instancia -Query $Consulta |
>> Export-Excel -Path $RutaExcelPrincipal -WorksheetName "VolumenDiario" -AutoSize -ClearSheet
Invoke-Sqlcmd : El comando 'Invoke-Sqlcmd' se encontró en el módulo 'SQLPS', pero el módulo no pudo cargarse. Para
obtener más información, ejecute 'Import-Module SQLPS'.
En línea: 1 Carácter: 1
+ Invoke-Sqlcmd -ServerInstance $Instancia -Query $Consulta |
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (Invoke-Sqlcmd:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CouldNotAutoloadMatchingModule
PS C:\Users\Usuario>
PS C:\Users\Usuario> # 3. Almacenar copia en carpeta de respaldo

```

3. Justificación Técnica de la solución aplicada A diferencia de CSV, PowerShell no maneja Excel nativamente, por lo que se invoca el módulo externo ImportExcel (el cual utiliza la librería EPPlus sin requerir Microsoft Excel instalado en el servidor). Se utiliza el parámetro -ClearSheet para actualizar la pestaña "VolumenDiario" con los datos frescos. Finalmente, el cmdlet Copy-Item satisface el requerimiento de resguardar el archivo.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Ligereza de Infraestructura:** El uso de Export-Excel (módulo de terceros estándar en la industria) evita la pésima práctica de instalar la suite de Office (Excel.exe) en un servidor de bases de datos de producción para generar reportes.

TEMA 4. Monitoreo Proactivo

Proyecto 1:

1. Enunciado del Ejercicio Crear un procedimiento almacenado que consulte diariamente la cantidad de trámites pendientes y envíe un correo electrónico con el resumen.

2. Script de la solución en SQL

```
254 | --4.1
255 |
256 | USE GradosTitulos;
257 | GO
258 |
259 | CREATE OR ALTER PROCEDURE dbo.SP_NotificarTramitesPendientes
260 | AS
261 | BEGIN
262 |     SET NOCOUNT ON;
263 |     DECLARE @Cantidad INT;
264 |     DECLARE @CuerpoMensaje NVARCHAR(MAX);
265 |
266 |     -- 1. Contar trámites pendientes (Asumiendo IdEstado = 1)
267 |     SELECT @Cantidad = COUNT(*) FROM Tramite.Tramite WHERE IdEstadoTramite = 1;
268 |
269 |     -- 2. Formatear el mensaje
270 |     SET @CuerpoMensaje = 'Estimado equipo, al día de hoy existen ' + CAST(@Cantidad AS NVARCHAR(10)) + ' trámites pendientes de revisión en el sistema.';
271 |
272 |     -- 3. Enviar correo usando Database Mail
273 |     EXEC msdb.dbo.sp_send_dbmail
274 |         @profile_name = 'PerfilCorreoInstitucional', -- Debe estar configurado previamente en SQL
275 |         @recipients = 'coordinacion@institucion.edu',
276 |         @subject = 'Alerta Diaria: Resumen de Trámites Pendientes',
277 |         @body = @CuerpoMensaje,
278 |         @body_format = 'TEXT';
279 | END;
280 | GO
```

3. Justificación Técnica de la solución aplicada El script hace uso de la función de agregación COUNT para obtener la volumetría de la tabla. Posteriormente, se invoca el procedimiento extendido del sistema sp_send_dbmail de la base de datos msdb. Este procedimiento encola el mensaje internamente en Service Broker, lo que significa que el envío del correo es asíncrono y no ralentiza la base de datos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Formatos Claros:** Enviar métricas pre-calculadas en el texto del correo evita que el usuario tenga que abrir archivos adjuntos o ejecutar consultas manuales, facilitando la toma de decisiones gerenciales.

Proyecto 2:

1. Enunciado del Ejercicio Registrar automáticamente en una tabla de auditoría todos los errores producidos durante la ejecución del procedimiento:

Tramite.SP_RegistrarTramite.

2. Script de la solución en SQL

```

53
54 USE GradosTitulos;
55 GO
56
57 -- Modificación del procedimiento existente
58 CREATE OR ALTER PROCEDURE Tramite.SP_RegistrarTramite
59     @IdEstudiante INT,
60     @TipoTramite VARCHAR(50)
61 AS
62 BEGIN
63     SET NOCOUNT ON;
64     BEGIN TRY
65         -- Lógica de negocio principal (Simulación de inserción)
66         INSERT INTO Tramite.Tramite (IdEstudiante, TipoTramite, FechaInicio)
67         VALUES (@IdEstudiante, @TipoTramite, GETDATE());
68     END TRY
69     BEGIN CATCH
70         -- Captura de funciones de error del sistema
71         INSERT INTO dbo.TablaAuditoriaErrores (
72             ProcedimientoOrigen,
73             NumeroError,
74             MensajeError,
75             LineaError,
76             FechaIncidente
77         )
78         VALUES (
79             ERROR_PROCEDURE(),
80             ERROR_NUMBER(),
81             ERROR_MESSAGE(),
82             ERROR_LINE(),
83             GETDATE()
84         );
85         -- Opcional: Lanzar el error hacia la aplicación
86         THROW;
87     END CATCH
88 END;
89 GO

```

función integrada ERROR_LINE() RETURNS int

3. Justificación Técnica de la solución aplicada El bloque TRY...CATCH es el estándar en T-SQL para el manejo de excepciones. Cuando ocurre un fallo en el TRY (por ejemplo, una violación de clave foránea), el control pasa inmediatamente al CATCH. Dentro del CATCH, las funciones del sistema como ERROR_MESSAGE() y ERROR_LINE() retienen el contexto del fallo original, permitiendo guardarlos permanentemente en una tabla física (TablaAuditoriaErrores).

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Persistencia Analítica:** Capturar el error en una tabla de base de datos en lugar de solo enviarlo a la aplicación cliente permite al DBA realizar análisis de tendencias (ej. "el error de FK ocurre siempre a fin de mes") sin depender de los logs de los programadores de software.

Proyecto 3:

1. Enunciado del Ejercicio Diseñar un sistema que monitoree el crecimiento del archivo de log de la base de datos y genere una alerta cuando supere el 80% del tamaño configurado.

2. Script de la solución en SQL

```

321 --4.3
322
323 USE GradosTitulos;
324 GO
325
326 CREATE OR ALTER PROCEDURE dbo.SP_MonitoreoLogSpace
327 AS
328 BEGIN
329     SET NOCOUNT ON;
330     DECLARE @PorcentajeUsado FLOAT;
331     DECLARE @Mensaje NVARCHAR(255);
332
333     -- 1. Consultar DMV para obtener porcentaje de uso del Log
334     SELECT @PorcentajeUsado = (used_log_space_in_bytes * 100.0) / total_log_size_in_bytes
335     FROM sys.dm_db_log_space_usage;
336
337     -- 2. Evaluar umbral (80%)
338     IF @PorcentajeUsado > 80.0
339     BEGIN
340         SET @Mensaje = 'ALERTA CRÍTICA: El archivo de LOG de GradosTitulos ha superado el 80% de su capacidad. (Uso actual: ' + CAST(ROUND(@PorcentajeUsado, 2), NVARCHAR(10)) + '%)';
341
342         -- 3. Generar Alerta
343         EXEC msdb.dbo.sp_send_dbmail
344             @profile_name = 'PerfilCorreoInstitucional',
345             @recipients = 'dba_urgencias@institucion.edu',
346             @subject = 'Servidor DB - Espacio en Log Critico',
347             @body = @Mensaje;
348     END
349 END;
350 GO
351 -- Este procedimiento debe acoplarse a un SQL Server Agent Job
352 -- para que se ejecute cada hora, por ejemplo.

```

3. Justificación Técnica de la solución aplicada Se consulta la Vista de Administración Dinámica (DMV) sys.dm_db_log_space_usage, que es muy ligera y no genera contención en disco al leer los tamaños en bytes. Al dividir el espacio usado entre el total, se obtiene la métrica exacta. Se integra con sp_send_dbmail condicionado a un bloque IF de forma que la alerta solo se dispara cuando se rompe la barrera del 80%.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Monitoreo Proactivo:** Alertar al 80% otorga al Administrador de Base de Datos un margen de tiempo crucial del 20% para intervenir (ej. ejecutando un backup de log para truncarlo) antes de que el servidor se detenga abruptamente por falta de espacio en disco (Error 9002).

Proyecto 4:

1. Enunciado del Ejercicio Implementar un sistema que registre automáticamente el tiempo de ejecución de los procedimientos almacenados principales y genere un ranking de rendimiento.

2. Script de la solución en SQL

```

54 --4.4
55
56 USE GradosTitulos;
57 GO
58
59 -- Consulta para extraer el ranking usando DMVs del sistema de caché
60 SELECT TOP 20
61     d.object_id,
62     OBJECT_NAME(d.object_id, d.database_id) AS ProcedimientoAlmacenado,
63     d.execution_count AS CantidadEjecuciones,
64     -- Convertir microsegundos a milisegundos para legibilidad
65     CAST(d.total_worker_time / 1000.0 AS DECIMAL(10,2)) AS TiempoTotal_CPU_ms,
66     CAST((d.total_worker_time / d.execution_count) / 1000.0 AS DECIMAL(10,2)) AS Promedio_CPU_ms,
67     CAST((d.total_elapsed_time / d.execution_count) / 1000.0 AS DECIMAL(10,2)) AS Promedio_DuracionRea
68     d.last_execution_time AS UltimaEjecucion
69 FROM sys.dm_exec_procedure_stats d
70 WHERE d.database_id = DB_ID('GradosTitulos')
71 -- Ordenar por el que consume más recursos en promedio
72 ORDER BY Promedio_CPU_ms DESC;
73 GO

```

object_id	ProcedimientoAlmacenado	CantidadEjecuciones	TiempoTotal_CPU_ms	Promedio_CPU_ms	Promedio_DuracionReal_ms	UltimaEjecucion
-----------	-------------------------	---------------------	--------------------	-----------------	--------------------------	-----------------

3. Justificación Técnica de la solución aplicada En lugar de crear tablas de monitoreo manuales (que añadirían lentitud al insertar un registro de inicio y fin dentro de cada SP), se utiliza la DMV sys.dm_exec_procedure_stats. Esta vista del sistema acumula automáticamente las estadísticas de rendimiento de los procedimientos mientras están en la memoria caché (Plan Cache) de SQL Server, ofreciendo un ranking de rendimiento en tiempo real y sin costo computacional adicional.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Mínima Interferencia (Observer Effect):** Monitorear el rendimiento con herramientas nativas de caché evita ralentizar el mismo sistema que se intenta medir, a diferencia de los engorrosos métodos de rastreo (Traces) manuales.

Proyecto 5:

1. Enunciado del Ejercicio Desarrollar un sistema integral de monitoreo que realice: monitoreo de bloqueos, detección de consultas lentas, identificación de CPU, registro en tablas históricas, envíos de alertas consolidadas diarias. (Debe integrar T-SQL, Agent, DMV, Mail, CATCH, transacciones).

2. Script de la solución en SQL

```

377 USE master;
378 GO
379
380 CREATE OR ALTER PROCEDURE dbo.SP_MonitoreoIntegral_Consolidado
381 AS
382 BEGIN
383     SET NOCOUNT ON;
384     BEGIN TRY
385         DECLARE @CuerpoHtml NVARCHAR(MAX) = '<h2>Reporte de Estado Consolidado</h2>';
386
387         -- 1. Identificar Sesiones con alto consumo de CPU (> 5000ms)
388         INSERT INTO GradosTitulos.dbo.HistoricoMonitoreo (TipoEvento, Sesion, Detalles, Fecha)
389         SELECT 'Alto CPU', session_id, 'CPU Time: ' + CAST(cpu_time AS VARCHAR), GETDATE()
390         FROM sys.dm_exec_requests WHERE cpu_time > 5000;
391
392         -- 2. Identificar Bloqueos Activos (Blocking)
393         INSERT INTO GradosTitulos.dbo.HistoricoMonitoreo (TipoEvento, Sesion, Detalles, Fecha)
394         SELECT 'Bloqueo', session_id, 'Bloqueado por: ' + CAST(blocking_session_id AS VARCHAR), GETDATE()
395         FROM sys.dm_exec_requests WHERE blocking_session_id <> 0;
396
397         -- 3. Consolidar el reporte diario de las tablas históricas para enviarlo
398         SET @CuerpoHtml = @CuerpoHtml + '<p>Incidentes registrados hoy: ' +
399         CAST((SELECT COUNT(*) FROM GradosTitulos.dbo.HistoricoMonitoreo WHERE CAST(Fecha AS DATE) = CAST(GETDATE() AS DATE))
400
401         -- 4. Envío de Correo mediante sp_send_dbmail
402         EXEC msdb.dbo.sp_send_dbmail
403         @profile_name = 'PerfilCorreoInstitucional',
404         @recipients = 'admin_bd@institucion.edu',
405         @subject = 'Reporte Consolidado de Monitoreo Integral',
406         @body = @CuerpoHtml,
407         @body_format = 'HTML';
408
409     END TRY
410     BEGIN CATCH
411         PRINT 'Error critico en el sistema de monitoreo: ' + ERROR_MESSAGE();
412     END CATCH
413 END;
414 GO

```

3. Justificación Técnica de la solución aplicada El script aborda un sistema integral centralizado. Utiliza sys.dm_exec_requests para capturar en tiempo real las métricas críticas del motor (CPU y Bloqueos). Estos datos transitorios se vuelcan inmediatamente mediante INSERT a una tabla física (HistoricoMonitoreo) para retención a largo plazo. Finalmente, el procedimiento engloba los hallazgos en formato HTML y los envía, todo recubierto en un bloque TRY...CATCH para no afectar el Agente

de SQL Server si la red de correo falla.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Arquitectura de Consolidación:** En sistemas empresariales, es una mala práctica enviar un correo individual por cada mini-incidente de CPU o bloqueo (provocando *Alert Fatigue* o fatiga de alertas). En su lugar, registrar silenciosamente en tablas históricas y generar un **resumen consolidado** diario es el estándar de oro en la administración de servidores de base de datos.